

COL703/7203: Logic for Computer Science (Jul-Nov 2026)

Horn-SAT, 2-SAT, DPLL

Kumar Madhukar

madhukar@cse.iitd.ac.in

Sept. 21 and 24, 2026

Horn formulas

- a **literal** is a boolean variable or its negation
- for a variable x , we have a **positive literal** (x) and a **negative literal** $(\neg x)$
- a **horn clause** is a finite disjunction of literals with **at most one positive literal**
- a **horn formula** is a finite conjunction of horn clauses
- **example**
$$(x \vee \neg y \vee \neg z \vee \neg w) \wedge (\neg x \vee \neg y \vee \neg w) \wedge (\neg x \vee \neg z \vee w) \wedge (\neg x \vee y) \wedge (x) \wedge (\neg z) \wedge (\neg x \vee \neg y \vee w)$$

Horn-SAT

- if the formula contains a **unit clause**, say (ℓ)
 - all **clauses** containing (ℓ) **is removed**
 - from all clauses containing $(\neg \ell)$ have $(\neg \ell)$ **removed**
- this may generate new unit clauses, which are propagated similarly
- if there are no unit clauses left, the formula can be **satisfied by setting every remaining variable to false**
- formula is **unsat** if propagation generates an **empty clause**

Horn-SAT example

$$(x \vee \neg y \vee \neg z \vee \neg w) \wedge (\neg x \vee \neg y \vee \neg w) \wedge (\neg x \vee \neg z \vee w) \wedge (\neg x \vee y) \wedge (x) \wedge (\neg z) \wedge (\neg x \vee \neg y \vee w)$$

Horn-SAT exercise

$$(x \vee \neg y \vee \neg z \vee \neg w) \wedge (\neg x \vee \neg y \vee \neg w) \wedge (\neg x \vee \neg z \vee w) \wedge (x) \wedge (\neg z) \wedge (\neg x \vee \neg y \vee w)$$

2-SAT

- given a 2-CNF formula, is it satisfiable or not
- every clause has 2 literals
- example
 $(\neg x \vee y) \wedge (\neg y \vee z) \wedge (x \vee \neg z) \wedge (z \vee y)$

2-SAT satisfiability check

- create a graph with $2n$ vertices (for a formula with n variables)
- corresponding to positive and negative literals for every variable
- for every clause $(a \vee b)$ (where a and b are literals), create directed edges $\neg a \rightarrow b$ and $\neg b \rightarrow a$

2-SAT satisfiability check

- create a graph with $2n$ vertices (for a formula with n variables)
- corresponding to positive and negative literals for every variable
- for every clause $(a \vee b)$ (where a and b are literals), create directed edges $\neg a \rightarrow b$ and $\neg b \rightarrow a$
- **claim:** if the graph contains a path from α to β , then it also contains a path from $\neg\beta$ to $\neg\alpha$

2-SAT satisfiability check

- create a graph with $2n$ vertices (for a formula with n variables)
- corresponding to positive and negative literals for every variable
- for every clause $(a \vee b)$ (where a and b are literals), create directed edges $\neg a \rightarrow b$ and $\neg b \rightarrow a$
- **claim:** if the graph contains a path from α to β , then it also contains a path from $\neg\beta$ to $\neg\alpha$
- **claim:** a 2-CNF formula is unsat iff there exists a variable x such that:
 - there is a path from x to $\neg x$
 - there is a path from $\neg x$ to x

2-SAT satisfying assignment

- pick an unassigned literal ℓ , with no path from ℓ to $\neg\ell$
- assign true to ℓ and all vertices reachable from ℓ (and assign false to their negations)
- repeat until all vertices are assigned

Example

$$(\neg x \vee y) \wedge (\neg y \vee z) \wedge (x \vee \neg z) \wedge (z \vee y)$$

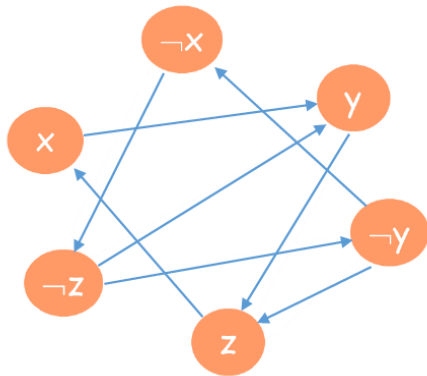


image source: <https://www.iitg.ac.in/deepkesh/CS301/assignment-2/2sat.pdf>

Exercise

$$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_3) \wedge (x_1 \vee x_2) \wedge (x_4 \vee \neg x_3) \wedge (x_4 \vee \neg x_1)$$

Tseitin transformation

- we know that an arbitrary boolean formula can be converted to CNF
- using De Morgan's law and distributivity property
- but this may result in an exponential explosion of the formula
- example: $(x_1 \wedge y_1) \vee (x_2 \wedge y_2) \vee \dots \vee (x_n \wedge y_n)$
- Tseitin transformation is guaranteed to only linearly increase the size of the formula

Question

Why does the conversion of $(x_1 \wedge y_1) \vee (x_2 \wedge y_2) \vee \dots \vee (x_n \wedge y_n)$ to CNF result in an exponential blow-up (in the formula size)?

Tseitin transformation: Example

(source: Wikipedia)

Consider the following formula ϕ .

$$\phi := ((p \vee q) \wedge r) \rightarrow (\neg s)$$

Consider all subformulas (excluding simple variables):

$$\neg s$$

$$p \vee q$$

$$(p \vee q) \wedge r$$

$$((p \vee q) \wedge r) \rightarrow (\neg s)$$

Introduce a new variable for each subformula:

$$x_1 \leftrightarrow \neg s$$

$$x_2 \leftrightarrow p \vee q$$

$$x_3 \leftrightarrow x_2 \wedge r$$

$$x_4 \leftrightarrow x_3 \rightarrow x_1$$

Conjoin all substitutions and the substitution for ϕ :

$$T(\phi) := x_4 \wedge (x_4 \leftrightarrow x_3 \rightarrow x_1) \wedge (x_3 \leftrightarrow x_2 \wedge r) \wedge (x_2 \leftrightarrow p \vee q) \wedge (x_1 \leftrightarrow \neg s)$$

All substitutions can be transformed into CNF, e.g.

$$\begin{aligned} x_2 \leftrightarrow p \vee q &\equiv (x_2 \rightarrow (p \vee q)) \wedge ((p \vee q) \rightarrow x_2) \\ &\equiv (\neg x_2 \vee p \vee q) \wedge (\neg(p \vee q) \vee x_2) \\ &\equiv (\neg x_2 \vee p \vee q) \wedge ((\neg p \wedge \neg q) \vee x_2) \\ &\equiv (\neg x_2 \vee p \vee q) \wedge (\neg p \vee x_2) \wedge (\neg q \vee x_2) \end{aligned}$$

ϕ and $T(\phi)$

- ϕ satisfiable implies $T(\phi)$ satisfiable?
- $T(\phi)$ satisfiable implies ϕ satisfiable?
- Are ϕ and $T(\phi)$ logically equivalent?

Question

How many new variables does the Tseitin transformation introduce?

Question

How do we know that the Tseitin transformation increases the formula size only linearly?

1-SAT to 3-SAT

$$c = (\ell)$$

1-SAT to 3-SAT

$$c = (\ell)$$

$$c' = (\ell \vee u \vee v) \wedge (\ell \vee \neg u \vee v) \wedge (\ell \vee u \vee \neg v) \wedge (\ell \vee \neg u \vee \neg v)$$

c' is satisfiable iff c is satisfiable.

2-SAT to 3-SAT

$$c = (\ell_1 \vee \ell_2)$$

2-SAT to 3-SAT

$$c = (l_1 \vee l_2)$$

$$c' = (l_1 \vee l_2 \vee u) \wedge (l_1 \vee l_2 \vee \neg u)$$

c' is satisfiable iff c is satisfiable.

$k(>3)$ -SAT to $(k-1)$ -SAT

$$c = (\ell_1 \vee \ell_2 \vee \dots \vee \ell_k)$$

$k(>3)$ -SAT to $(k-1)$ -SAT

$$c = (\ell_1 \vee \ell_2 \vee \dots \vee \ell_k)$$

$$c' = (\ell_1 \vee \ell_2 \vee \dots \vee \ell_{k-2} \vee u) \wedge (\ell_{k-1} \vee \ell_k \vee \neg u)$$

c' is satisfiable iff c is satisfiable.

breaking 3SAT similarly to get 2SAT fails!

$$c = (\ell_1 \vee \ell_2 \vee \ell_3)$$

breaking 3SAT similarly to get 2SAT fails!

$$c = (l_1 \vee l_2 \vee l_3)$$

$$c' = (l_1 \vee l_2 \vee u) \wedge (l_3 \vee \neg u) \quad (\text{still 3!})$$

breaking 3SAT similarly to get 2SAT fails!

$$c = (l_1 \vee l_2 \vee l_3)$$

$$c' = (l_1 \vee l_2 \vee u) \wedge (l_3 \vee \neg u) \quad (\text{still 3!})$$

$$c' = (l_1 \vee u) \wedge (l_2 \vee l_3 \vee \neg u) \quad (\text{still 3!})$$

Question

Is it possible to convert a 3-SAT formula to a *logically equivalent* 2-SAT formula, if we can add as many variables and clauses as we want?

What if we only want an *equisatisfiable* 2-SAT formula?

Davis-Putnam Algorithm

- **unit-clause** – if there is a unit clause ℓ , delete all clauses containing ℓ , and delete all occurrences of $\neg\ell$ from other clauses
- **pure-literal** – if there is a pure literal ℓ , delete all clauses containing ℓ
- **eliminate a variable by resolution** – choose an atom p and perform all possible resolutions on clauses that clash on p and $\neg p$. Add these resolvents to the set of clauses and then delete all the clauses containing p or $\neg p$.
- use these repeatedly; but use resolution only if the the first two rules do not apply

Davis-Putnam Algorithm

- if **empty clause** is produced, the formula is **unsat**
- if no more rules are applicable, report **sat**
- why does this terminate?
- why is this correct?
- example: $(p) \wedge (\neg p \vee q) \wedge (\neg q \vee r) \wedge (\neg r \vee s \vee t)$
- **exercise**: $(\neg a \vee b) \wedge (a \vee c) \wedge (\neg c \vee d) \wedge (\neg d \vee b \vee \neg e) \wedge (\neg b \vee \neg c) \wedge (e)$

DPLL Algorithm

- creating all possible resolvents is very inefficient
- DPLL improves on the DP algorithm by replacing the variable elimination with a search for a model of the formula

- Davis-Putnam-Logemann-Loveland algorithm (about 60 years old)
- combines search and deduction to decide satisfiability of CNF formulas
- based around backtrack search for a satisfying valuation

DPLL Algorithm

- the **state** of the algorithm is a pair $(\mathcal{F}, \mathcal{A})$
- a state is **successful** if $\mathcal{A}$ sets some literal in each clause of $\mathcal{F}$ to true
- a **conflict** state is one where $\mathcal{A}$ sets every literal in some clause to false
- let $\mathcal{F}|_{\mathcal{A}}$ denote the formula that we get after **simplifying** $\mathcal{F}$ using $\mathcal{A}$
- $(\mathcal{F}, \mathcal{A})$ is a **conflict** state if $\mathcal{F}|_{\mathcal{A}}$ contains the empty clause $\square$
- $(\mathcal{F}, \mathcal{A})$ is a **successful** state if $\mathcal{F}|_{\mathcal{A}}$ is the empty set of clauses

DPLL Algorithm

1. initialize $\mathcal{A}$ to be an empty assignment
2. while there are unit clauses $\{\ell\}$, add $\ell \mapsto 1$ to $\mathcal{A}$
3. if $(\mathcal{F}, \mathcal{A})$ is a successful then stop and output $\mathcal{A}$
4. if $(\mathcal{F}, \mathcal{A})$ is a **conflict** state then apply **clause learning** to get a new clause $\mathcal{C}$
 - if $\mathcal{C}$ is $\square$ then stop and output *unsat*
 - add $\mathcal{C}$ to $\mathcal{F}$; backtrack to the highest level at which $\mathcal{C}$ is a unit clause; goto 2
5. add a new decision assignment $p_i \mapsto 1/0$ to $\mathcal{A}$; goto 2

Example

$C_1: \{\neg p_1, \neg p_4, p_5\}$

$C_2: \{\neg p_1, p_6, \neg p_5\}$

$C_3: \{\neg p_1, \neg p_6, p_7\}$

$C_4: \{\neg p_1, \neg p_7, \neg p_5\}$

$C_5: \{p_1, p_4, p_6\}$

$\mathcal{A}: \langle p_1 \mapsto 1, p_2 \mapsto 0, p_3 \mapsto 0, p_4 \mapsto 1 \rangle$

unit propagation generates a sequence of implied assignments: $\langle p_5 \xrightarrow{C_1} 1, p_6 \xrightarrow{C_2} 1, p_7 \xrightarrow{C_3} 1 \rangle$

conflict: C_4 becomes false!

Learned clause

if clause learning gives a clause $\mathcal{C}$, then we would want

- $\mathcal{F} \equiv \mathcal{F} \cup \mathcal{C}$
- $\mathcal{C}$ should be a conflict clause (must evaluate to false under the current assignment)
- all variables in $\mathcal{C}$ should be decision variables (fixed using decision assignments)

Correctness

- **termination** – a sequence of decisions leading to a conflict cannot be repeated
- **correctness** – if empty clause is learned, then $\mathcal{F}$ is unsatisfiable (because $\mathcal{F} \equiv \mathcal{F} \cup \mathcal{C}$)

Clause learning

$\mathcal{A}$: $\langle p_1 \mapsto 1, p_2 \mapsto 0, p_3 \mapsto 0, p_4 \mapsto 1, p_5 \xrightarrow{C_1} 1, p_6 \xrightarrow{C_2} 1, p_7 \xrightarrow{C_3} 1 \rangle$

$A_8 := \{\neg p_1, \neg p_7, \neg p_5\}$ (clause C_4)

$A_7 := \{\neg p_1, \neg p_5, \neg p_6\}$ (resolve A_8, C_3)

$A_6 := \{\neg p_1, \neg p_5\}$ (resolve A_7, C_2)

$A_5 := \{\neg p_1, \neg p_4\}$ (resolve A_6, C_1)

$\vdots$

$A_1 := \{\neg p_1, \neg p_4\}$

Clause learning

$\mathcal{A}$: $\langle p_1 \mapsto 1, p_2 \mapsto 0, p_3 \mapsto 0, p_4 \mapsto 1, p_5 \xrightarrow{C_1} 1, p_6 \xrightarrow{C_2} 1, p_7 \xrightarrow{C_3} 1 \rangle$

$A_8 := \{\neg p_1, \neg p_7, \neg p_5\}$ (clause C_4)

$A_7 := \{\neg p_1, \neg p_5, \neg p_6\}$ (resolve A_8, C_3)

$A_6 := \{\neg p_1, \neg p_5\}$ (resolve A_7, C_2)

$A_5 := \{\neg p_1, \neg p_4\}$ (resolve A_6, C_1)

$\vdots$

$A_1 := \{\neg p_1, \neg p_4\}$

the algorithm backtracks to the decision level $\langle p_1 \mapsto 1 \rangle$ (which is the highest level at which the learned conflict clause becomes a unit clause)

Clause learning

$\mathcal{A}$: $\langle p_1 \mapsto 1, p_2 \mapsto 0, p_3 \mapsto 0, p_4 \mapsto 1, p_5 \xrightarrow{C_1} 1, p_6 \xrightarrow{C_2} 1, p_7 \xrightarrow{C_3} 1 \rangle$

$A_8 := \{\neg p_1, \neg p_7, \neg p_5\}$ (clause C_4)

$A_7 := \{\neg p_1, \neg p_5, \neg p_6\}$ (resolve A_8, C_3)

$A_6 := \{\neg p_1, \neg p_5\}$ (resolve A_7, C_2)

$A_5 := \{\neg p_1, \neg p_4\}$ (resolve A_6, C_1)

$\vdots$

$A_1 := \{\neg p_1, \neg p_4\}$

the algorithm backtracks to the decision level $\langle p_1 \mapsto 1 \rangle$ (which is the highest level at which the learned conflict clause becomes a unit clause)

what about the things that were desirable from a learned clause?

Thank you!